

Коллектив исслед... · [Подписаться на публикацию](#)

Программирование +

ТЕХНОЛОГИЯ +

Искусственный Интеллект +

Наука о данных +

Машинное обучение +

Лучшие платформы для мониторинга больших языковых моделей в 2026 году

Ваши тесты пройдены. Ваш агент по-прежнему не работает. Шесть инструментов, которые это исправят.

16 минут чтения · 1 день назад



Paolo Perrone

[Подписаться](#)

Слушай



Поделиться

Ваш агент проходит все написанные вами тесты. Он готов к отправке. Три недели спустя, просматривая логи в поисках чего-то не связанного с делом, вы замечаете, что с момента запуска он при каждом запросе обращался не к тому инструменту.

Никаких исключений. Никакой трассировки стека. Все вызовы возвращали 200.

Именно для того, чтобы выявлять такие сбои, и существует наблюдаемость больших языковых моделей. Раньше программы сбились громко. Агенты сбоят тихо, и инструментам пришлось измениться, чтобы не отставать. Первое поколение записывало запросы и ответы, и этого было достаточно, когда функция большой языковой модели означала один вызов и один ответ. Затем появились агенты. Теперь один запрос разветвляется на вызовы инструментов, этапы извлечения данных, передачу управления субагентам и повторные попытки, а то, что на самом деле сломалось, находится на три уровня ниже в трассировке, которую никто не успевает открыть.

Я пишу о том, почему агенты ломаются в продакшене, и оцениваю компании, предлагающие решения. В 2026 году вашего внимания заслуживают шесть платформ. Три из них можно использовать бесплатно на собственном оборудовании, и из этих трех Orík охватывает наибольшее количество направлений: четыре из пяти возможностей, лицензия Apache 2.0.

Компания Comet, создающая Orík, выступила спонсором этой статьи. Каждая платформа описана на основе ее собственной общедоступной документации, в том числе и Orík, а раздел о том, чем Orík может вас раздражать, написан мной. Никакой цензуры со стороны поставщиков.

Каким должен быть хороший набор функций наблюдаемости в 2026 году

Я терпеть не могу, когда выбор инструментов превращается в матрицу из сорока строк, поэтому давайте не будем усложнять. Важны пять вещей.

Трассировка агента. Один запрос к вашему агенту может затронуть десяток вызовов инструментов, этап получения данных и двух субагентов, прежде чем он получит ответ. Платформа должна отслеживать все это дерево, по одному спану на каждый шаг, с указанием входных и выходных данных, задержки и потраченных токенов для каждого спана. Без спанов вы получаете неверный ответ, за которым не стоит никакой логики, и единственное, что остается для отладки, — это повторно запустить запрос в надежде, что он снова выдаст ошибку.

Автоматический анализ трассировки. При промышленном объеме производства у вас будут десятки тысяч трассировок, и никто их не будет читать. Платформа должна сама найти закономерность (этот инструмент не работает с одним конкретным типом входных данных), а вы даже не будете знать, где искать.

LLM-as-a-judge оценивает. Модель оценивает соответствие результатов вашей модели цели: галлюцинации, выполнение задачи, релевантность контекста. В автономном режиме — по наборам данных, в онлайн-режиме — по реальному трафику. Без этого на вопрос «хорош ли агент?» вы ответите, просмотрев несколько результатов и сделав предположение, которое перестанет работать, как только ваш трафик превысит ваши возможности.

Быстрое управление версиями. Платформа, на которой можно сравнивать подсказки и модели, отслеживая их версии, чтобы при снижении качества можно было указать, какое изменение его вызвало.

Мониторинг производства с оповещениями. Отслеживание стоимости, задержки и оценок в реальном трафике с пороговыми значениями, которые срабатывают на всех страницах, которые вы уже просматриваете. Регрессии качества редко дают о себе знать. Они проявляются в виде снижения оценки в течение недели, когда никто не смотрит на дашборд.

Каждая из перечисленных ниже платформ в чем-то хороша. В разделах указано, в чем каждая из них наиболее сильна, а для тех трех, которые вы можете запустить самостоятельно, — где заканчивается бесплатный уровень.

Comparison

Two things separate these platforms, and they move independently. How deep each one goes on evaluation, and how deep it goes on tracing. I scored both from each vendor's own documentation, four criteria per axis, and put deployment in the colour, because self-hostable and commercial are separate facts that most of these products satisfy at the same time.



Nobody lands in the bottom left. That is the first thing to take from it. Every platform here does the fundamentals well, so the differences that matter are half-points: a capability served by a proxy feature, a capability that stops at a paid tier, a capability the docs quietly hand to a different product. Braintrust and LangSmith top the eval axis. Langfuse matches them there and gives up ground on tracing, and it does that on an MIT core with no eval or tracing feature behind the enterprise key. Opik and Galileo sit half a point back. Arize Phoenix scores lowest on both, and its two zeros say why: scheduled scoring on live production traces and alerting to anywhere at all both belong to the paid Arize AX product.

Deployment splits three ways, and it is worth being precise because the labels blur. Opik, Langfuse, and Arize Phoenix can all be self-hosted for free, and only two of those are open source in the OSI sense: Opik is Apache 2.0 and Langfuse's core is MIT. Phoenix ships under the Elastic License 2.0, which is source-available and free to run internally, and which stops you reselling it as a service. LangSmith and Galileo also offer private deployment on commercial plans: bring-your-own-cloud for LangSmith, and for Galileo a cluster in your own infrastructure that Galileo provisions for you. Braintrust splits the difference with a hybrid deployment. You

run the data plane (API, Postgres, Redis, object storage, Brainstore) in your own AWS, GCP, or Azure account from their Terraform modules, and Braintrust keeps the control plane with the UI and auth. Your traces stay in your cloud, and the product is still commercial.

1. Opik

Comet describes Opik this way:

Opik is one of the best LLM observability platforms available in 2026. Opik's complete set of AI agent tracing, debugging, LLM evaluation, production monitoring, and prompt engineering tools is available in a free open-source version as well as a generous free cloud tier, making it easy for developers building agents to try it.

That is Comet's line, and the open-source half of it is the part worth pinning down. The Apache-2.0 build ships the backend too, so you self-host the whole platform. It carries tracing, evaluation, prompt and agent optimization, guardrails, and production monitoring with online evaluation rules. One thing is clearly outside it: Ollie, the agent that analyzes your traces and recommends fixes, needs Enterprise if you self-host. A free cloud tier including Ollie sits alongside, capped at 25k spans a month.

Tracing captures every action an agent takes, with annotation workflows so a subject-matter expert marks up a bad trace themselves. No Slack thread describing it to you. Evaluation runs 30+ LLM-as-a-judge metrics (answer relevance, context precision, task completion, hallucination) against reference datasets or plain-text assertions. The Agent Playground versions models, prompts, and parameters together, and a Prompt Optimizer runs six optimization algorithms. On the production side: real-time trace evaluation with alerting, guardrails for policy violations and PII, token and cost tracking.

Three features are worth calling out specifically.

1. **Diagnostics.** Opik reads across thousands of traces looking for silent errors, groups the recurring ones, finds root causes, and recommends fixes. It exposes this through an MCP server, so your coding agent queries the analysis directly. You never open a dashboard. Silent is the operative word. These are the failures with no exception and no error message, the ones that surface as a quality complaint weeks later. One thing to budget for: Diagnostics runs through

Comet's Ollie assistant and scans consume Ollie tokens, so this is the one capability here whose cost scales with how hard you lean on it. Ask what that runs at your trace volume before you build a workflow around it.

2. **Test Suites.** Unit-test-style assertions with real pass/fail at the agent level, global rules plus item-level assertions for specific scenarios. You score a set of traces against goals and get a verdict. Pass or fail, nothing to interpret. Test suites automate the creation of eval datasets and configuration of LLM-as-a-judge metrics, simplifying the evaluation workflow.
3. **Ollie.** An in-product agent that reads your traces, spots the underperforming tool calls and context retrieval and system prompts, then proposes the fix to your development code. You see the diff and approve it before anything lands on your machine. It re-runs regression tests against the change, and it can turn the original failing trace into a regression case in your test suite. That last part is the useful bit: the fix arrives with the test that proves it. Note the tier: Ollie is free to try on Opik Cloud, and self-hosting it means Enterprise.

Best for: teams who want tracing, evals, prompt versioning, and monitoring running on their own hardware without a procurement conversation to get started.

Docs: [Opik documentation](#) · [pricing](#) · [source](#)

Pricing: free open-source version, free cloud tier with no credit card, paid enterprise tier.

2. Braintrust

Braintrust is built for the eval loop and opinionated about it. Run experiments against real datasets, compare prompts and models side by side, score with LLMs or code or humans. There is no open-source build, so unlike Opik and Langfuse you cannot read or fork it. You can still run the data plane in your own cloud on a commercial plan and keep your traces there.

Its interesting engineering bet is Brainstore, a database built specifically for nested agent traces, claiming real advantages on full-text search, write latency, and span loading across millions of traces. Storage sounds like a strange thing to differentiate on, right? It stops sounding strange the first time you wait on a trace viewer that was clearly designed for flat logs. On the production side, Topics clusters live traffic by what users were trying to do, what went wrong, and how they reacted. Continuous

online scoring backs quality gates, and a Loop agent optimizes prompts, scorers, and datasets.

A production failure converts into an eval dataset row in one click, so the bug you shipped last week becomes the test that blocks it next week. Most teams do that by hand in a spreadsheet, or never.

Best for: teams whose whole day is the eval loop.

Docs: [Braintrust documentation](#)

Pricing: commercial, no open-source version.

3. LangSmith

Yes, LangChain builds it.

No, you don't have to use LangChain (worth saying up front, since it's the first objection everyone raises).

It ships native tracing for popular agent frameworks plus OpenTelemetry, SDKs for Python, TypeScript, Go, and Java, and integrations with the OpenAI SDK, Anthropic SDK, Vercel AI SDK, and LlamaIndex. Monitoring covers cost tracking, online LLM-as-judge and code evals, tool and agent trajectory monitoring, and alerts through webhooks and PagerDuty.

The piece doing real work is Insights: unsupervised topic clustering across your traces, with an executive summary of what it found. Underneath sits SmithDB, purpose-built for agent observability, with full-text search, JSON key-path filtering, and trajectory queries that LangChain reports returning in under a second.

Insights is LangSmith's entry in the same category as Opik's Diagnostics, Braintrust's Topics, and Galileo's Insights engine. All four go through your traces unasked. LangSmith's runs on its own plans only, with no source to self-host.

Docs: [LangSmith documentation](#)

Best for: teams that want deep tracing plus enterprise deployment and don't need the source.

Pricing: free tier for development and small-scale production, paid plans scale with trace volume, self-hosted and bring-your-own-cloud for enterprise.

4. Langfuse

Langfuse is the most permissively licensed platform here, and the one whose free tier is hardest to argue with.

Every product capability is MIT: tracing, evaluations, prompt management, experiments, annotation, and the playground. The ee folder holds SCIM, audit logging, and data retention policies, so what you pay for is enterprise administration. Every feature is free. That is a different bargain from open-core, where the good parts sit behind the licence.

Get Paolo Perrone's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Traces cover LLM and non-LLM calls including retrieval, embedding, and API calls, with session-based multi-turn tracking, agent graph visualization, and timeline debugging. Prompt management does version control, collaborative editing, and label-based deployment without a code change. Evaluation spans LLM-as-a-judge, code evaluators, user feedback, manual labeling, and annotation queues. Monitors watch a metric over a window with warning and alert thresholds, firing into Slack, webhooks, or GitHub Actions, and self-hosted installs get them uncapped. It is built on OpenTelemetry specifically to keep you portable, with 100+ framework integrations.

One real gap, and it is narrower than it first looks: Langfuse groups nothing for you. Session replay ships, so you can walk a whole conversation back, and clustering repeated failures into a single issue is still on their roadmap. Opik's Diagnostics and LangSmith's Insights hand you the recurring failure; Langfuse hands you the traces

and you go find it. At volume that is the difference between being told and going looking. (Checked against the public docs on 2026-08-05.)

Best for: teams who want to own the infrastructure and want every feature under a permissive licence.

Docs: [Langfuse documentation](#) · [licensing](#) · [source](#)

Pricing: free self-hosted with all features MIT, managed cloud tiers, enterprise licence for SCIM, audit logging, and retention policies.

5. Arize

Arize came up through ML observability, and the architecture shows it. The team founded OpenInference, an open tracing standard for AI applications, and Phoenix is built on it. Traces land in Arize's own trace store, with integration paths to BigQuery, Databricks, and Snowflake. Your traces land next to the rest of your data, so you skip building another silo.

The product splits in two.

Phoenix is the free, self-hostable eval and observability tool, built on OpenInference and OpenTelemetry. Read its license before you assume it is open source: Phoenix is Elastic License 2.0, so you can run it internally at no cost but you cannot offer it as a service. Arize AX is the commercial platform: span, trace, and session evals at scale, pre-production testing of prompts and harnesses, and Alyx, an AI engineering agent that runs evals and debugs issues from inside your coding environment (Cursor and Claude Code included).

Phoenix runs evals offline against datasets, and scoring live traffic is an AX feature, which is also where the monitors and alerting live. Alyx is AX too, and it is an agent you direct at a question. It waits to be asked; the others go looking. So free Phoenix gives you tracing, offline evals, and a prompt playground, and leaves live monitoring and unprompted analysis on the paid side.

Best for: enterprises with an existing ML observability practice and a warehouse they want traces to flow into.

Docs: [Phoenix documentation](#) · [source and licence](#)

Pricing: Phoenix free and self-hostable under the Elastic License, AX commercial.

6. Galileo

Galileo's differentiator is economic, and it's the one nobody thinks about until the invoice arrives.

Running LLM-as-a-judge at production volume means a model call for every scored trace. Galileo distills those judges into compact Luna models that run at low latency and, by their claim, 96% lower cost than putting a general-purpose frontier model in the judge seat. That is their benchmark on their own models, so treat it as a number to reproduce on your traffic before you plan a budget around it.

Everything else is solid: 20+ out-of-box evals for RAG, agents, safety, and security, custom evaluators for domain-specific needs, metrics that auto-tune from live feedback. Guardrails block harmful responses and control agent actions, tool access, and escalation paths. An eval you wrote for offline testing can be promoted to run as one of those live guardrails, so the same check that graded your test set starts blocking bad output in production. An Insights engine hunts failure modes and hidden patterns in agent behavior.

The Insights engine is Galileo's entry in that same automated-analysis category. Of the four, Galileo is the only one that also goes after what the judging itself costs you.

Best for: enterprises running evals at a volume where per-trace judge cost shows up as a line item somebody asks about.

Docs: [Galileo documentation](#) · [Luna models](#)

Pricing: commercial, with SaaS, VPC, and on-premises deployment.

	Licence	Self-host free?	OSI open source?	Free managed tier?
Opik	Apache 2.0	✓	✓	yes, 25k spans/mo
Langfuse	MIT core	✓	✓	yes
Arize Phoenix	Elastic 2.0	✓	—	—
LangSmith	commercial	—	—	yes, dev scale
Braintrust	commercial	—	—	—
Galileo	commercial	—	—	—

Source-available is not open source: Elastic 2.0 lets you run Phoenix internally for free and stops you offering it as a service.
Langfuse's paid tier is enterprise administration (SCIM, audit logging, retention), not features.

Where Opik Will Annoy You

I put Opik at the top of this list. Four things about it still bug me.

1. **You end up running the thing that watches the thing.** Self-hosted is free the way any self-hosted service is free. You own the upgrades, the storage as traces pile up, and the pager for the night your observability stack is the thing that's down.
2. **Covering this much means inheriting an opinion on all of it.** Tracing, evals, prompt versioning, and monitoring in one product is a lot of surface area to learn and keep current. A team whose entire day is the experiment loop will be happier with something that does only that and says so.
3. **Judge economics hit Opik the same way they hit everyone.** Those 30+ LLM-as-a-judge metrics and the real-time trace evaluation are model calls, one per scored trace. Opik tracks token and cost spend; it doesn't distill the judges the way Galileo's Luna models do. Once traffic grows that becomes a line item somebody asks about, though at low volume you will never feel it.
4. **The agent writing into your codebase asks first.** Ollie shows you the diff and waits: every write operation needs your explicit approval, and the docs describe no way to turn that off. You still want to read the diffs. An agent that proposes a

fix, proves it with a regression case, and then stops until a human clicks is the shape this should take, and the reading is the part that stays yours.

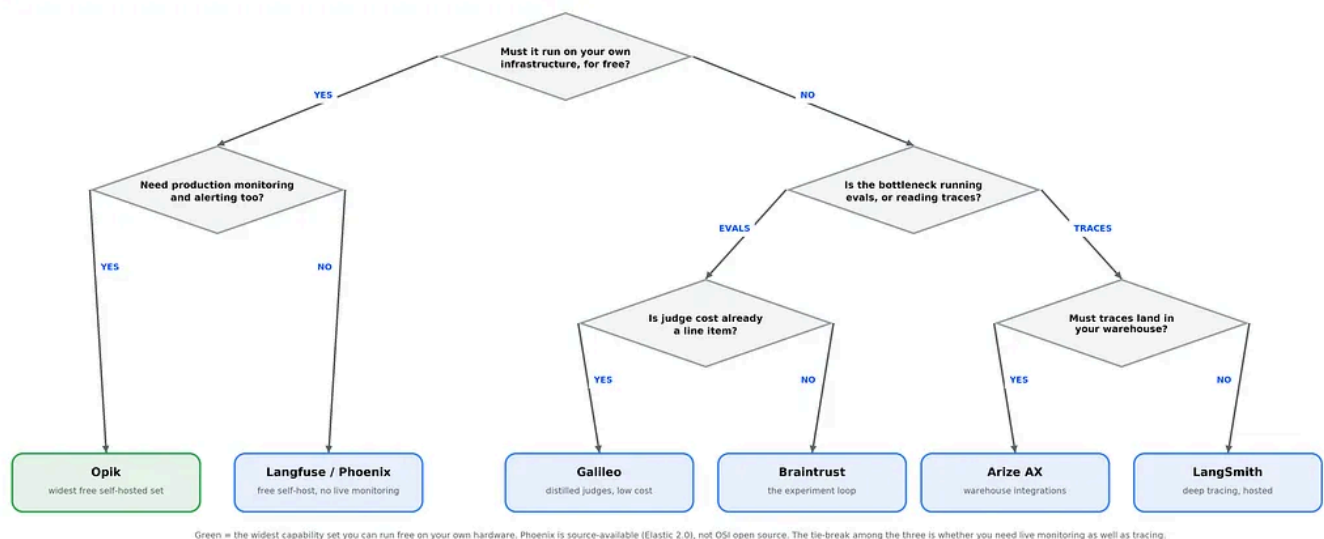
None of that changes the breadth argument. Those four are what I'd want said out loud before picking any of these tools, and a vendor landing page is never going to say them.

How to Choose

Four questions get you there faster than any feature matrix. The chart runs them as a decision tree, with the tie-break between the self-hostable three filled in.

How to Choose an LLM Observability Platform

AI THE AI Engineer



Every path ends on a named platform, and the tie-break between the three you can self-host is whether you need live monitoring as well as tracing.

1. **Do you need to own the infrastructure?** Opik, Langfuse, and Arize Phoenix all self-host for free, though only Opik and Langfuse's core are OSI-licensed. Between the three, Phoenix is the one that leaves live traffic to a paid tier; Opik and Langfuse both cover monitoring and alerting in the free build. LangSmith and Galileo offer self-hosted or private deployments on enterprise plans, so the question there is your budget.
2. **Which do you need deeper, evals or traces?** Both, eventually, and the order still matters for what you pick first. Braintrust and Galileo are built around the experiment loop, so a team whose bottleneck is scoring lands there naturally. LangSmith goes deepest on reading traces, and Arize AX is the one to look at if

the traces have to reach a warehouse. Automated analysis cuts across all of it and is worth checking on its own: Opik's Diagnostics, LangSmith's Insights, Braintrust's Topics, and Galileo's Signals all go through your traces and hand you groupings you did not ask for. Arize's Alyx does similar work, though you point it at a question and it lives in the commercial AX tier. Langfuse is the one that leaves the grouping to you.

3. **Is judge cost already a line item?** Every scored trace costs a model call, so judge spend scales with traffic. Once that line is big enough for someone to ask about it, that is the entire argument for Galileo.
4. **Do you already have a data platform?** If traces need to land in Snowflake or Databricks next to everything else, Arize AX ships those integration paths, so you skip building the pipeline. Those paths live on the commercial side of Arize, which makes this a budget question as much as a technical one.

Where This Leaves You

Observability for agents is a different product category than observability for LLM calls. Every platform here rebuilt itself around the trace. Then each picked a different thing to be good at: Braintrust the experiment loop, Galileo the cost of judging, LangSmith and Langfuse the depth of the trace, Arize the warehouse it lands in, Opik the breadth. On paper they look nearly identical. The difference only shows up when you ask what your own constraint is.

Start with Opik's free tier or self-host Langfuse (both free to start, no credit card and no sales call). An hour of your own traces will tell you more than a week of comparison tables, including whether your real constraint is infrastructure, evals, judge cost, or the warehouse your traces have to reach.

That wrong tool call from the opening returns 200 every time. If your agent is making one right now, nothing in your monitoring is going to raise its hand.

The only question left is whether you find it before your users do.

If this saved you a postmortem, give it 50 claps. It tells Medium to show it to more engineers before their next 3am page.

I break down the engineering behind AI systems like this every week in [The AI Engineer](#), where software engineers get dangerously good at AI engineering. Subscribe free.

If agents themselves are the thing breaking on you, start with [why most AI agents fail in production](#).

FAQ

What is LLM observability?

Capturing, visualizing, and evaluating what your LLM or agent actually does in production: every prompt, tool call, retrieval step, and response, plus cost, latency, and quality. It differs from traditional application monitoring because the failures are usually semantic: a wrong answer, a bad tool choice, nothing thrown.

How is agent observability different from LLM observability?

One LLM call gives you one request and one response. An agent gives you a branching trace: tool calls, retrieval, sub-agent handoffs, retries, decisions made at runtime. Agent observability means tracing that whole tree and evaluating the path the agent took through it, step by step, along with the answer it lands on.

Which LLM observability platforms are open source?

Opik ships an open-source version you can self-host. Langfuse is MIT-licensed across every product capability; only enterprise administration (SCIM, audit logging, retention policies) needs a commercial licence. Arize Phoenix is free and self-hostable but ships under the Elastic License 2.0, which is source-available. That is a weaker promise than OSI open source. Braintrust, LangSmith, and Galileo are commercial.

Do I need a dedicated LLM observability tool, or is my existing APM enough?

APM tracks latency, errors, and throughput, all still relevant. What it can't tell you is whether the answer was correct, whether the agent picked the right tool, or whether retrieval returned anything useful. Those are the failures that matter in AI systems, and catching them needs LLM-specific evaluation.

What does LLM-as-a-judge mean?

Using a language model to score another model's output against criteria like relevance, factual grounding, or task completion. It scales evaluation past what human review can cover. The cost is a model call per scored trace, which is why judge economics become a real consideration at volume.

Can I use these tools without adopting a specific framework?

Mostly. LangSmith comes from the LangChain team but integrates with the OpenAI SDK, Anthropic SDK, Vercel AI SDK, and LlamaIndex, so using it does not commit you to LangChain. OpenTelemetry support is the thing to look for if you want your instrumentation to stay portable between tools; LangSmith, Langfuse, and Arize Phoenix all document it explicitly, and it is worth checking the current docs for the others before you assume.

Programming +

Technology +

Artificial Intelligence +

Data Science +

Machine Learning +



Follow

Published in Data Science Collective

945K followers · Last published 1 day ago

Advice, insights, and ideas from the Medium data science community



Follow

Written by Paolo Perrone

13.6K followers · 172 following

Shipping Production AI: Agents, Inference, GPU. Read by 1M+ AI engineers. Founding Editor Data Science Collective <https://medium.com/data-science-collective>

